Technical Design Document (TDD)

1. Status

Proposed logical design based on the PRD. Confirm the actual repository, Electron version, SDK, database, browser driver and packaging configuration before implementing changes.

2. Proposed stack

- Desktop shell: Electron + TypeScript.
- UI: React + TypeScript.
- Preload: narrow, typed bridge; no unrestricted IPC exposure.
- Validation/contracts: existing Zod schemas/shared TypeScript types where present.
- Orchestration: main-process service with bounded concurrency, timeout, cancellation and structured states.
- Portal interaction: isolated Electron sessions/browser panes and existing approved driver/CDP approach.
- Parsing: pure TypeScript parser modules, separate from UI/browser interaction.
- OCR: worker-based OCR only for permitted cases where DOM/accessibility data is unavailable; package and test assets.
- Local persistence: SQLite with versioned migrations.
- Credentials: OS-backed secure storage (e.g., Electron safeStorage where suitable); fail safely if unavailable.
- PDF: HTML/CSS template rendered using Electron printToPDF or the existing validated PDF service; test font embedding/rupee symbol.
- Tests: existing test runner (Vitest per task list), fixture tests, integration tests, end-to-end smoke.
- Packaging: existing Electron Builder or configured packager; separate Windows/macOS builds; signing/notarization as applicable.

This is a target architecture, not a claim that every item is already in the repository.

3. Component boundaries

Renderer

Customer/member forms, insurer selection, progress board, results, manual entry, history, settings, PDF preview/export. No direct Node.js, database, filesystem or secret access.

Preload bridge

Explicitly exposes approved methods. Never expose raw `ipcRenderer` or arbitrary channel names.

Main process

Owns privileged services: quote orchestration, session manager, adapter registry, storage, SecretStore, PDF generation, diagnostics and lifecycle.

PaneManager/Driver

Creates isolated insurer sessions, navigates to approved origins, handles waits/popups/screenshots and cleans up. No CAPTCHA/MFA bypass.

Orchestrator

Starts adapters, applies timeouts/retry/cancel, records state transitions, preserves partial results, emits sanitized progress events.

Adapter registry

Maps stable insurer IDs to adapters; validates inputs/outputs against shared schemas.

Result normalizer/comparison

Normalizes premiums and provenance; compares only valid comparable values.

Storage and PDF

SQLite holds quote data and metadata. SecretStore holds credentials. PDF renderer consumes validated data and approved branding.

4. Main flow

1. Advisor creates quote.
2. UI validates and calls preload API.
3. Main process validates again and persists draft.
4. Advisor selects insurer sessions and completes required login/OTP/CAPTCHA interaction.
5. Orchestrator starts supported workflows.
6. Adapter navigates permitted flow and parses result.
7. Shared schema validates output; result persists.
8. UI receives sanitized progress.
9. Advisor reviews results and manual fallbacks.
10. PDF service renders validated stored data.
11. User exports PDF; quote history records artifact reference.

5. Reliability rules

- One adapter failure cannot cancel other adapters.
- Do not retry unsafe/non-idempotent steps blindly.
- Retry only a documented transient error once.
- Persist terminal states and validated results.
- Unknown errors become sanitized INTERNAL_ERROR with correlation ID.
- UI remains responsive during OCR/portal work.

6. Security boundaries

Renderer is untrusted; insurer content is untrusted external content; main process is privileged. Use context isolation, disabled Node integration in renderer, strict IPC allowlists, input validation, navigation/popup policy and restrictive CSP for app-owned UI.

7. Key decisions to confirm

Electron version and supported OS floor; browser view/CDP implementation; SQLite driver/migrations; OCR worker; PDF engine/fonts; updater hosting/signing; code signing ownership; exact adapter scope and portal authorization.